

SGP PREPARATION

CHARUSAT Online Course Chatbot - End-to-End RAG Architecture Report

Branch: main | Platform: FastAPI + React + ChromaDB + MongoDB

1. Project Overview

The CHARUSAT Online Course Assistant is a Retrieval-Augmented Generation (RAG) chatbot that answers questions about CHARUSAT's online degree programmes (Online BBA, BCA, MBA, MCA): fees, eligibility, curriculum, semesters, credits, admissions, examinations and policies.

- Knowledge sources: official university website (charusat.online) scraped into Markdown + official programme PDFs (Programme & Placement Reports).
- Pipeline: scrape / extract --> chunk --> embed --> store in vector DB --> retrieve --> rerank --> LLM answer.
- Persistence: MongoDB stores users, conversations and message history; guest chats stay in the browser session.

2. Current Vector Database Stats (main branch)

Measured directly from backend/vector_db/chroma.sqlite3 on branch main:

- | | |
|-------------------------|---|
| Total chunks | 389 |
| Web-scraped (md) | 237 chunks (13 Markdown source files) |
| PDF chunks | 152 chunks (5 PDFs) |
| Embedding size | 384 dimensions (BAAI/bge-small-en-v1.5) |
- Web (md): bba-admission-data (17), bca-admission-data (19), ciqa (24), home (21), privacy-policy (16), terms-conditions (66), feedback (1), mandatory-disclosures (2), contact (4), online_bba (16), online_bca (17), online_mba (17), online_mca (17).
 - PDF: PPR Online BCA (47), PPR Online MCA (40), PPR Online MBA (33), PPR Online BBA (29), Fees Refund Policy (3).

3. Technology Stack and Why It Was Chosen

Frontend

React 19 + Tailwind 4 + Vite Modern component UI, fast dev/build, dark/light theme, no heavy UI library.
react-router-dom 7 Routing for /chat, /login, /admin, /settings with conversation URLs /chat/{id}.

Backend

FastAPI + uvicorn Async Python API; auto OpenAPI docs; simple single-dyno deployment on Render.
LangChain (core/community) Document model, Chroma integration, RecursiveCharacterTextSplitter.
Chromadb (langchain-chroma) Local file-based vector DB - zero external service, 389 vectors, perfect for this scale; persists to backend/vector_db/.

Models

Embeddings: BAAI/bge-small-en-v1.5 (384-dim) loaded through FastEmbed ONNX - 80-120 MB RAM vs 500+ MB for PyTorch; fallback all-MiniLM-L6-v2.
LLM (primary): Groq API - openai/gpt-oss-120b with failover openai/gpt-oss-20b, qwen/qwen3.8-27b, allam-2-7b. Fast, free-tier, cheap inference.
LLM (config tier): Gemini models (gemini-3.6-flash etc.) are configured in PREFERRED_MODELS for a Gemini deployment path.
Fallback: Deterministic local answer generator - extracts exact claim values using a configurable attribute taxonomy, never hallucinates and never dumps raw chunks.

Data / Ingestion

Web scraping: requests + BeautifulSoup (lxml) - fetches HTML, removes nav/footer/scripts, renders semantic Markdown per page. crawl4ai is listed in requirements but the active scraper uses bs4.
PDF extraction: PyMuPDF (fitz) - extracts tables via find_tables() and page text while excluding table regions; cleans encoding artefacts.
Tokenization: tiktoken (used internally for splitter length estimation).

Database

MongoDB Atlas (pymongo) db = charusat_assistant; collections users, conversations, messages. Free serverless tier, optional - bot still works without it.

4. RAG Flow - From User Question to Answer

Example question: "What is the total fee for Online MBA?"

1. User types in React Chat.jsx -> ChatComposer calls handleSend() -> chatService.sendMessage() -> apiFetch() POST /api/chat with {question, conversation_id, history} and JWT bearer token.
2. Backend chat_routes.chat_message() runs. Small talk/greetings are answered conversationally without RAG. For real questions it gets the shared RAGChain from api.py.
3. Retriever.retrieve() analyses the query: intent = programme (Online MBA detected), general, or programme_overview; extracts attribute keywords and 2-3 word phrases.
4. Chroma similarity_search_with_relevance_scores pulls a candidate pool (k=20) filtered to program_name='online_mba' + general/policy sources; verbatim-phrase chunks are merged in.
5. Each candidate is scored = semantic score + bounded bonuses (keyword, value proximity, verbatim phrase, source authority), capped at 0.95; phrase-matched chunks are guaranteed above the 0.52 floor.
6. Near-duplicate chunks removed (Jaccard > 0.82), diversity capped at 2 chunks per (source,page), final TOP_K = 6 selected.
7. Conflict detection scans the pool for conflicting numeric claims (e.g. fee differences across PDFs, window 160 chars) using the attribute taxonomy.

8. Context is built: each chunk gets a header (Source, Page, Chunk ID, Content Type, Programme, Score) and content trimmed (800 chars, tables up to 3400, total 5600 chars). A Conflict Notice is prepended if any.
9. build_prompt() wraps the SYSTEM_PROMPT + context + question. llm_client.generate() calls Groq (temperature 0.2, timeout 10s, max 1024 tokens) and strips internal references (file paths, pages, scores) from the answer.
10. Backend returns {answer, conversation_id, message_id, sources (PDF citations only), suggested_questions}. Chat.jsx renders the assistant bubble, stores it, and navigates to /chat/{id}.
11. Authenticated turns are persisted to MongoDB before/with the response.

5. How Chunks Are Created (Chunking Methods)

A. Website pages (ingestion.py)

- `crawl_internal_pages()` BFS crawls from `https://charusat.online/` (max 250 pages, 0.5s delay), saves each page as Markdown with frontmatter (Title, URL, Category, Last Scraped).
- `load_documents()` keeps only .md files that have a URL metadata; derives `program_name` from the URL path.
- `split_documents()` uses `LangChain RecursiveCharacterTextSplitter` with `CHUNK_SIZE=600` chars, `overlap=100`, separators `["\n\n", "\n", " ", ""]` so paragraphs stay intact.
- Each chunk gets metadata: `source`, `source_filename`, `page_title`, `url`, `category`, `program_name`, `document_type`, and a per-file `chunk_id` starting at 1.

B. PDF pages (pdf_ingestion.py)

- `PDFIngestor` opens with `PyMuPDF`, page by page. Tables are extracted first via `page.find_tables()` and converted to compact Markdown tables with empty columns/rows removed.
- Text is extracted while EXCLUDING regions that overlap detected table bounding boxes (>50% overlap).
- Text segments are cleaned (encoding artefacts, page footer "1 | Page", triple newlines).
- Chunking: text uses `PDF_CHUNK_SIZE=800` chars; table segments use `PDF_TABLE_CHUNK_SIZE=3600` chars to preserve wide table rows; `overlap=100` for both.
- Metadata: `source='pdfs/<file>'`, `filename`, 1-based page number, `content_type=text/table`, `program_name` (regex on filename), `chunk_id`.

C. Embedding and storage

- `EmbeddingStore.build_vector_store()` -> `Chroma.from_documents()` writes 384-dim float vectors into `chroma.sqlite3` with an HNSW index (`space=l2`, `ef_construction=100`, `max_neighbors=16`).
- `rebuild_vectordb.py` rebuilds from scraped md only; `ingest_pdfs.py` ADDS PDF chunks without destroying existing data (deletes only re-ingested filenames to avoid duplicates).

6. Chat History Storage in MongoDB

Database `charusat_assistant`, three collections:

users	{_id, name, email (unique), password_hash, google_id, avatar, role, created_at, updated_at}
conversations	{_id, user_id (indexed with updated_at), title (auto from first question), created_at, updated_at}
messages	{_id, conversation_id, role (user/assistant), content, sources[], suggested_questions[], created_at}

Flow: first question creates a conversation (title = snippet of question) -> user message inserted -> assistant message inserted with its sources and follow-up chips -> `conversation.updated_at` refreshed. Guest chats are NOT persisted; they live in `sessionStorage` and are migrated to Mongo after sign-in via `POST /api/chat/migrate`.

7. Why These Particular Choices (Rationale)

- RAG over pure LLM: answers are grounded in official fee/eligibility documents - reduces hallucination and keeps data current without model fine-tuning.
- `bge-small` + `ONNX`: tiny memory (fits free Render dyno), 384-dim is fast for search, and quality is excellent for English university text.
- `ChromaDB` local: no SaaS/API cost, no network dependency, `LangChain`-native; easy to migrate later.
- `Groq` first: ultra-low latency and free tier; the tiered list auto-fails over on 429/errors and finally drops to the deterministic extractor so the bot ALWAYS answers.
- Source-authority tiers: official PDFs (`PPR_Online`) outrank boilerplate (terms/privacy/home), and programme-matched chunks get a bonus - fixes wrong/weak retrieval.

- Conflict detection + numeric safety prompts: prevents the LLM from silently picking one of two different fee values or doing arithmetic the document does not state.
- Phrase boosting + candidate pooling: rescues small correct chunks (e.g. "placement assistance") buried below legal boilerplate in similarity ranking.

8. What Can Be Improved

- Hybrid retrieval: add BM25/keyword search and fuse with embeddings (RRF) - currently dense + phrase regex only.
- Cross-encoder reranker (e.g. bge-reranker / Cohere Rerank) over the candidate pool to improve TOP_K ordering beyond the hand-tuned bonus weights.
- Better LLM default: use Gemini (gemini-2.5/3.x) as PRIMARY for reasoning quality with Groq as failover; today Gemini code exists but Groq runs first.
- Streaming responses (SSE/WebSocket) + typing indicator for a smoother UX.
- Multilingual / Indic support: switch embeddings to bge-m3 or text-embedding-3-large and add Gujarati/Hindi answers.
- Multimodal RAG: index fee-schedule IMAGES inside PDFs using vision models (documented as a future content_type=image).
- Evaluation harness (RAGAS / golden set) to measure retrieval hit-rate and faithfulness on new scrapes.
- Query expansion / rewrite: rewrite pronouns and context using the last 2 turns for multi-turn retrieval.
- Vector DB for scale: migrate to Qdrant/Pinecone/Milvus when chunk count or concurrency grows; keep Chroma for now (389 chunks).
- Feedback loop: store thumbs-up/down + user corrections in MongoDB and feed them into prompt improvement / re-ingestion.
- Scraper upgrade: use crawl4ai/Playwright for JavaScript-heavy pages instead of requests+bs4 only.
- Caching: memoize repeated questions at the API layer and add per-query latency metrics for monitoring.

9. Future Roadmap

1. Phase A - Quality: reranker + hybrid retrieval + RAGAS eval harness; migrate primary LLM to Gemini flash family with Groq failover and streaming.
2. Phase B - Product: multilingual answers, PDF vision/multimodal indexing, admin dashboard to manage knowledge base and trigger re-ingestion, feedback buttons.
3. Phase C - Scale: vector DB migration (Qdrant), caching layer, conversation summarization for long context, WhatsApp/Telegram/voice channel integration.
4. Phase D - Operations: monitoring (latency, retrieval quality, fallback rate), auto re-scrape scheduler with drift detection, A/B prompt testing.

Verified facts: 389 total chunks (237 md + 152 pdf) read live from chroma.sqlite3 on branch main. Stack, flows, chunking and storage described from backend/app/ and frontend/src/*.*